Guion: pruebas de Barbería Norte

Guion para explicar las cuatro pruebas del proyecto: integración, unitarias, extremo a extremo y carga. Cada bloque trae lo que se dice y los comandos para ejecutarlo.

La raíz del repositorio es:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci
```

En esta máquina el entorno de Python ya está creado en `backend/.venv` con Python 3.12. Hay que activarlo antes de `pytest` o de `uvicorn`.

---

0. Apertura

Qué decir

Barbería Norte es una aplicación para reservar citas. Tiene tres partes: la pantalla en React, el servidor en FastAPI y la base de datos PostgreSQL. Las pruebas revisan cada parte por separado y también el recorrido completo. Voy a correr cuatro tipos: integración, unitarias, extremo a extremo y carga.

---

1. Pruebas de integración

Qué decir

Estas pruebas levantan un PostgreSQL real, dentro de Docker, con Testcontainers. No usan la base que queda en el puerto 5432: cada corrida crea la suya, aplica las migraciones y la tira al terminar.

Revisan que el servidor y la base trabajen juntos:

- Un cliente se registra, inicia sesión y consulta su perfil.
- Una reserva se guarda, y una segunda reserva en el mismo horario recibe conflicto.
- Un barbero no puede crear una cita como si fuera cliente.
- Cada barbero solo ve su agenda. Si pide la de otro, la API responde 403.

Son 4 pruebas, en `backend/tests/integration/`.

Antes de correrlas

Docker Desktop tiene que estar abierto. La aplicación no hace falta.

Comandos

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/backend
source .venv/bin/activate
pytest -q tests/integration
```

Qué decir al ver el resultado

Si termina en verde, las cuatro pruebas pasaron: el registro, el guardado de la cita, el choque de horario y los permisos de la agenda quedaron comprobados contra una base real.

---

2. Pruebas unitarias

Qué decir

Estas pruebas no levantan servidor ni base de datos. Revisan una regla a la vez, con datos fijos.

En el servidor hay 14, en `backend/tests/unit/test_domain.py`. Cubren:

- Horarios de lunes a sábado, de 09:00 a 18:00. El domingo no tiene espacios.
- Un corte de 30 minutos y uno de 60 ocupan bloques distintos.
- Dos citas seguidas no se pisan. Si se cruzan, se rechazan.
- La contraseña se guarda cifrada y el token de sesión se puede leer.
- No se acepta una cita en el pasado ni una fecha sin zona horaria.

En la pantalla hay 6, con Vitest. Cubren el formulario de registro y de entrada, y el formato de la hora en Ciudad de México. El sábado pasa al lunes como siguiente día hábil.

Comandos del servidor

Misma terminal, con el entorno ya activado:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/backend
source .venv/bin/activate
pytest -q tests/unit
```

Comandos de la pantalla

Otra terminal:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/frontend
npm test
```

Qué decir al ver el resultado

El servidor debe marcar 14 pruebas en verde. La pantalla debe marcar 6. Esas reglas ya están revisadas antes de probar el sistema completo.

Para ver las dos capas del servidor juntas, unitarias e integración:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/backend
source .venv/bin/activate
pytest -q
```

Eso son las 18 pruebas del servidor.

---

3. Pruebas de extremo a extremo

Qué decir

Cypress abre la aplicación como lo haría una persona. Aquí sí tienen que estar corriendo la base, el servidor y la pantalla.

Hay dos recorridos, en `e2e/cypress/e2e/booking.cy.js`:

1. Un cliente se registra, elige servicio, barbero y horario, confirma la cita y la cancela. En pantalla se ve Programada y después Cancelada.
2. La prueba crea una cita por la API a nombre de Cliente Agenda. Luego entra como Mateo Ruiz, el barbero, y esa cita aparece en su agenda.

Comandos para levantar el programa

Terminal 1, base de datos:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci
docker compose up -d
```

Terminal 2, API. Queda en http://localhost:8000 y la documentación en http://localhost:8000/docs:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/backend
source .venv/bin/activate
uvicorn app.main:app --reload
```

Terminal 3, pantalla. Queda en http://localhost:5173:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/frontend
npm run dev
```

Espera a que las tres estén listas. Si el puerto 8000 ya está ocupado, la API ya está corriendo y no hay que lanzar otra.

Comandos de Cypress

Terminal 4. Para ver el navegador mientras se explica:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/e2e
npx cypress open
```

En la ventana de Cypress elige E2E y el archivo `booking.cy.js`.

Para dejar el resultado en la terminal, sin ventana:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci/e2e
npx cypress run
```

Qué decir al ver el resultado

Deben pasar las 2 pruebas. El cliente recorrió la reserva de punta a punta, y el barbero vio en su agenda la cita que el cliente creó.

---

4. Pruebas de carga

Qué decir

k6 simula varias personas consultando la API al mismo tiempo. No usa la pantalla. Solo necesita la base y el servidor del bloque anterior.

El escenario corto es el mismo de GitHub Actions: 10 usuarios virtuales durante 30 segundos. Cada uno consulta los servicios, los barberos y sus propias citas. Antes, la prueba registra un cliente y entra con él.

La prueba falla si los errores llegan al 1 %, o si el 95 % de las respuestas tarda más de 500 milisegundos.

Comandos

Con la API en http://localhost:8000, desde la raíz del repositorio, no desde `e2e`:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci
k6 run load/api.js
```

Si se quiere un escenario más largo, sin tocar el archivo:

```
k6 run -e VUS=50 -e DURATION=2m load/api.js
```

Para explicar, usa el primero. Dura medio minuto.

Qué decir al ver el resultado

Al final, `http_req_failed` debe quedar por debajo de 1 % y `http_req_duration` p(95) por debajo de 500 ms. Esas dos líneas aparecen como umbrales superados.

---

5. Cierre

Qué decir

En local se vieron las cuatro capas: la base real con las pruebas de integración, las reglas sueltas con las unitarias, el recorrido de un cliente y de un barbero con Cypress, y la respuesta de la API cuando varias personas consultan a la vez.

GitHub Actions vuelve a correr las mismas pruebas en cada cambio. El archivo `.github/workflows/ci.yml` tiene cuatro checks: `backend`, `frontend`, `e2e` y `load`. Si uno falla, el cambio no queda integrado.

---

6. Cómo parar todo al terminar

En la terminal de la API y en la de la pantalla, `Ctrl+C`.

Después, desde la raíz:

```
cd /Users/fernandarivera/Desktop/QA/proyecto/test_suite_with_ci
docker compose down
```